PINKPAPER AI

AI-Powered Technical Blog Writer

Project Explanation & Technical Guide

Core idea
Turn a topic into a structured technical blog through a multi-step LangGraph workflow.

Workflow
Routing → optional research → planning → section writing → finalization

Stack
Python · LangGraph · LangChain · Groq LLM · Tavily Search · Streamlit

Live application: https://blog-gen-sidd.streamlit.app/

Prepared as a practical guide for understanding, explaining, and discussing the project in interviews.

1. Project Overview

PinkPaper AI Blog Writer is a web application that helps a user turn a topic into a structured technical blog post. Instead of asking a language model to write the entire article in one request, the application organizes the work into stages: decide whether research is needed, collect source material when appropriate, create an outline, write individual sections, and assemble the final article.

The project demonstrates a practical use of LLMs and agentic workflow orchestration. LangGraph controls the order of operations and passes shared state between nodes; Streamlit provides the user interface.

Project goal

Make technical blog creation more organized, research-aware, and easier to inspect than a single large prompt. The workflow separates planning from writing so each section has a defined goal, bullet points, word target, and citation/code requirements.

What it is—and is not

- It is an LLM-powered writing workflow with optional web research.
- It uses supplied search evidence to guide research-based claims and citations.
- It does not guarantee that every generated statement is factually correct; sources and citations should be checked before publishing.
- The current workflow is text-focused. The supplied graph definition contains no image-generation node.

2. Main Features

- **Topic-driven generation:** a user supplies a subject and requests a blog.
- **Routing:** the router selects a research mode such as closed-book, hybrid, or open-book.
- **Optional web research:** Tavily search returns results that are converted into evidence records.
- **Structured planning:** an LLM creates a Plan with a title, audience, tone, blog type, constraints, and tasks.
- **Section-based writing:** LangGraph sends each task to a worker that writes one Markdown section.
- **Finalization:** a final node combines generated sections into the finished blog.
- **Streamlit interface:** users can run the workflow and view/download the resulting Markdown, depending on the deployed UI implementation.

3. Architecture and Workflow

The workflow is built as a directed graph. Each node performs a focused responsibility, and the shared State carries information through the pipeline.

1. Router	2. Research (optional)
Chooses mode and whether research is needed.	Runs Tavily queries and stores validated evidence.
3. Orchestrator / Planner	**4. Worker fan-out**
Creates a structured Plan and writing tasks.	A worker writes each task as a Markdown section.
5. Finalize	**6. End**
Combines sections into the final article.	Returns the completed workflow result.

Graph flow

START → router → (research, if selected) → orchestrator → worker(s) → finalize → END

In the graph definition, the router's conditional edge chooses either the research node or the orchestrator. The research node then connects to the orchestrator. The orchestrator uses fanout() to dispatch each Plan task to the worker node. The workers' outputs are collected by the graph state and passed to finalization.

Why LangGraph?

- It represents the process as explicit nodes and edges instead of a single monolithic function.
- It supports conditional routing and parallel task dispatch through Send.
- It makes the workflow easier to debug because each stage has a distinct responsibility.
- Shared state keeps the topic, mode, evidence, plan, and generated sections available across steps.

4. Important Components

State

Carries workflow data such as topic, mode, queries, evidence, plan, timestamps, sections, and final output.

Router node

Determines whether research is needed and selects the workflow mode.

Research node

Calls Tavily for each query and maps useful search results into EvidenceItem records. The current safer version avoids an extra LLM structured-output call for evidence extraction.

Orchestrator node

Calls the LLM with the Plan schema to produce a structured outline with tasks.

fanout()

Uses LangGraph Send to create one worker task per planned section.

worker_node()

Builds a focused prompt from a task's goal, bullets, target length, code/citation flags, and selected evidence; returns the section Markdown.

finalize_blog

Combines worker sections into the final article. Exact behavior depends on the implementation in app/nodes/finalize.py.

5. Structured Data and Schemas

Pydantic schemas help define the shape of data passed between LLM stages. This reduces ambiguity and makes the workflow's expectations explicit.

Schema	Purpose	Key fields
Task	One section-writing assignment.	id, title, goal, bullets, target_words, tags, requires_research, requires_citations, requires_code
Plan	The overall blog blueprint.	blog_title, audience, tone, blog_kind, constraints, tasks
EvidenceItem	One research source.	title, url, published_at, snippet, source
RouterDecision	Research routing decision.	needs_research, mode, reason, queries, max_results_per_query
EvidencePack	A list of evidence records (defined in schemas).	evidence

6. Technology Stack

Technology	Role in this project
Python	Main programming language.
LangGraph	Graph workflow, conditional routing, state, and worker fan-out.
LangChain	Message abstractions and model invocation.
Groq LLM API	Provides language-model inference through the configured ChatGroq model.
Tavily Search	Retrieves web search results when the router selects research.
Pydantic	Defines and validates structured objects such as Plan and EvidenceItem.
Streamlit	Web UI for running the workflow and presenting the result.

7. Technical Walkthrough

A typical request follows this sequence:

- **Input:** the user enters a topic and chooses whether to use research, if exposed by the UI.
- **Routing:** the router produces a mode and research queries.
- **Research:** Tavily results are collected, cleaned, deduplicated, and mapped to evidence records.
- **Planning:** the orchestrator requests a structured Plan with 5–7 tasks, each containing bullets and requirements.
- **Fan-out:** each task is sent to a worker with the task details, topic, mode, plan summary, and relevant evidence.
- **Writing:** the worker generates one Markdown section and returns it with the task ID.
- **Finalization:** the final node assembles the sections and produces the article.

Simplified example of the graph

```
graph.add_edge(START, "router")
graph.add_conditional_edges("router", route_next, {
    "research": "research",
    "orchestrator": "orchestrator",
})
graph.add_edge("research", "orchestrator")
graph.add_conditional_edges("orchestrator", fanout, ["worker"])
graph.add_edge("worker", "finalize")
graph.add_edge("finalize", END)
```

Why section-by-section writing?

Each section gets a smaller, focused task instead of a full-article prompt. This can improve structure and makes the workflow easier to inspect. However, fan-out may run multiple model calls close together, so rate limits and total token usage must be considered.

8. Deployment and Configuration

- The deployed application is available at https://blog-gen-sidd.streamlit.app/.
- The app uses secrets/environment variables for API credentials. On Streamlit Community Cloud, configure required keys under the app's Secrets settings.
- Keep credentials out of source control. Do not commit .env files or paste API keys into public issues or chat.
- Confirm that the configured Groq model ID is available to the account and that the required dependencies are listed in the project's dependency file.
- After pushing changes to the connected GitHub branch, wait for Streamlit Cloud to finish redeploying before testing.

Typical configuration names

```
GROQ_API_KEY = "your_key_here"
GROQ_MODEL = "your_supported_model_id"
TAVILY_API_KEY = "your_key_here"
```

The exact environment-variable names should match the project's config.py and research.py implementations.

9. Challenges, Limitations, and Improvements

Engineering challenges encountered

- **Structured tool-call failures:** a model may return malformed tool-call arguments. The evidence path was simplified to map search results directly into validated records.
- **Rate limits:** multiple worker calls can exceed the provider's tokens-per-minute quota. Smaller prompts and bounded retries can help, but they do not override provider limits.
- **Research quality:** search results can be irrelevant, stale, or incomplete. Sources and dates need validation.
- **Output quality:** generated code, facts, and citations should be reviewed before publication.

Potential next improvements

- Add explicit concurrency limits or a queue for worker calls.
- Track token usage, request latency, and per-node failures in logs.
- Add robust retry/backoff that respects provider retry hints and avoids retry storms.
- Add a citation validation step to verify that cited URLs exist in the evidence set.
- Add unit tests for routing, evidence normalization, section ordering, and finalization.
- Expose article length and tone settings in the prompts/state so UI controls reliably affect generation.
- Cache research results where appropriate and add a clear message when research is unavailable.

10. Interview-ready Explanation

30-second pitch

“I built PinkPaper AI, a technical blog-writing application using Python, LangGraph, LangChain, Groq, Tavily, and Streamlit. Instead of generating a whole article in one call, LangGraph routes the request, optionally gathers web evidence, creates a structured outline, sends individual sections to writing workers, and combines them into a Markdown article. The project helped me learn stateful LLM workflows, structured outputs, research grounding, deployment, and API rate-limit handling.”

Questions you should be ready to answer

- **Why LangGraph instead of one LLM call?** It makes stages, routing, and task fan-out explicit and easier to maintain.
- **What does the router do?** It decides whether the topic needs research and chooses the workflow mode.
- **Why use Pydantic schemas?** They define expected fields for plans, tasks, and evidence and support validation.
- **What is fan-out?** It dispatches separate section tasks to worker nodes through LangGraph Send.
- **How do you handle rate limits?** Reduce prompt size and unnecessary evidence, stagger bounded retries, and limit concurrency if needed.
- **How do you reduce hallucinations?** Ground current claims in supplied evidence, cite sources, and review generated content. This reduces risk but does not eliminate it.

11. Final Summary

PinkPaper AI demonstrates how to combine a web interface, an LLM provider, web search, structured data models, and a graph-based workflow into a practical AI application. Its most important design idea is separation of responsibilities: routing, research, planning, writing, and finalization each have their own stage. The next step is to strengthen observability, rate-limit management, tests, and source verification as the application evolves.

Live demo: https://blog-gen-sidd.streamlit.app/

Note: This guide is based on the project structure and code shared in the conversation. Details of the frontend and finalization node may differ if those files have changed.